# GaussBridge Plan Walkthrough

Nov 5, 2026 · @Shivam Mittal

## How to review

We have two files that drive the project, and each teammate should read this walkthrough, then tick or change the decisions in the checklist below before we start on Monday, November 9.

| File | What it is | Who reads it |
|---|---|---|
| `CLAUDE.md` | The full project brief: what we build, how every stage works, exact tool versions and APIs, conventions, evaluation, literature. Claude Code loads it automatically in every session. | Every teammate once; every Claude Code agent always |
| `TASKS.md` | The task board: about 30 tasks split into three workstreams, each with dependencies, steps and a pass/fail acceptance check. | Each of us, to pick the next task |

**What we need from each person:** read sections 2 to 4 here (about 15 minutes), skim your own workstream in `TASKS.md`, then comment on anything you disagree with and tick the decisions you accept. Anything nobody objects to by Sunday night is treated as agreed.

## The project in plain words

We take two snapshots of a scene (before and after) and generate many realistic ways the robot and objects could have moved between them, as 3D Gaussian worlds we can film from any camera.

**How it flows, end to end:**

1. **Make training motions.** In the ManiSkill simulator, a motion planner solves StackCube and PickCube many times per starting scene, with randomised grasps, heights, waypoints and speeds. That gives about 20k varied example trajectories.

2. **Build the 3D appearance once.** We photograph the robot and cubes in many random poses from 36 virtual cameras and fit Gaussians, split into parts: each robot link, each cube, the table. Each part can then be moved on its own.
3. **Train the motion model.** A diffusion transformer learns to fill in robot joint angles and cube poses for every in-between step, given the start and end. The start and end are fixed inputs, so every generated motion lands exactly on the end state.
4. **Generate.** For a new start and goal, sample many candidates, push them away from collisions, force cubes to move only while held, throw out bad ones, keep the most different.
5. **Render.** Move each Gaussian part to its pose at each step and render from the robot's own cameras, with free labels (depth, flow, tracks).
6. **Prove it helps.** Train ManiSkill's Diffusion Policy on real demos alone vs. real plus our generated demos, and compare task success.

**The main result we want:** adding our generated demos raises policy success, and comes close to adding perfect planner demos. The planner needs full simulator knowledge; our method only needs the two snapshots.

## CLAUDE.md, section by section

`CLAUDE.md` has 16 sections; the ones to read closely are §0, §2, §4, §10 and §12, and the rest are reference material for whoever owns that stage.

| § | Section | What it says | Read closely? |
|---|---|---|---|
| 0 | TL;DR | Title, problem, core idea, v1 testbed (ManiSkill, Panda, StackCube + PickCube), the three novelty claims | Everyone |
| 1 | Ground rules for agents | Never guess an API, conventions are fixed, every task has a test, keep `TASKS.md` updated | Everyone (short) |
| 2 | System overview | The pipeline diagram and the three workstreams | Everyone |

| § | Section | What it says | Read closely? |
|---|---|---|---|
| 3 | Tech stack | Pinned versions (Python 3.11, ManiSkill 3.0.1, gsplat 1.5.3), install commands, Vulkan setup, Dockerfile, CHTC submit file and staging rules | Aditya (owns setup) |
| 4 | Conventions | Units, quaternion order (w,x,y,z), 6D rotations, OpenCV cameras, joint order, 20 Hz control, 128-step model time | Everyone: most bugs will come from breaking these |
| 5 | Verified facts | What we confirmed in ManiSkill and gsplat source and by running them: cameras per task, file layouts, planner API, grasp rule, Diffusion Policy data format, gsplat rasterizer details | Owner of each stage |
| 6 | Data | Planner randomisation knobs, dataset sizes and seed ranges, the episode file format, the model's motion tensor | Aditya, Shivam |
| 7 | Gaussian representation | Parts in their own frames, one reconstruction for robot + objects + table, how composition and view-dependent colour work, labels, render-quality checks | Amogh |
| 8 | Motion model | Transformer architecture, flow-matching training, sampling with guidance, the grasp-consistency fix, filters, baselines | Shivam |
| 9 | Export for Diffusion Policy | Exactly which keys the policy's data loader needs and how we fill them | Aditya, Amogh |

| § | Section | What it says | Read closely? |
|---|---|---|---|
| 10 | Evaluation | Motion metrics, render metrics, the downstream experiment arms, ablations | Everyone |
| 11 | Repo layout | Planned folders, modules and scripts | Everyone (skim) |
| 12 | Team plan | Owners, weekly plan, gates, what to cut if behind | Everyone |
| 13 | Risks | Nine risks with early warning signs and fixes | Everyone (skim) |
| 14 | Literature | Related work grouped by theme, and how we position against the closest papers | Everyone before writing the report |
| 15 | Open decisions | Items still to settle (deadline, staging, W&B) | Everyone |

**Three ideas in §7–§8 that everything else depends on:**

- **Parts, not frames.** We never rebuild the 3D scene per time step. Each Gaussian belongs to one rigid part (a link, a cube, the table), and a time step is just a set of part poses.
- **Endpoints are inputs, not targets.** The start and end are fed to the model clean and never noised, so samples hit the goal exactly instead of drifting toward it.
- **Objects move only when held.** After sampling, a cube's pose is recomputed from the gripper's pose while it is grasped, and it stays put otherwise. Samples that can't satisfy this are rejected.

## TASKS.md: how the task board works

`TASKS.md` splits the work into setup tasks plus three workstreams that run in parallel for two weeks and converge in weeks 3–4.

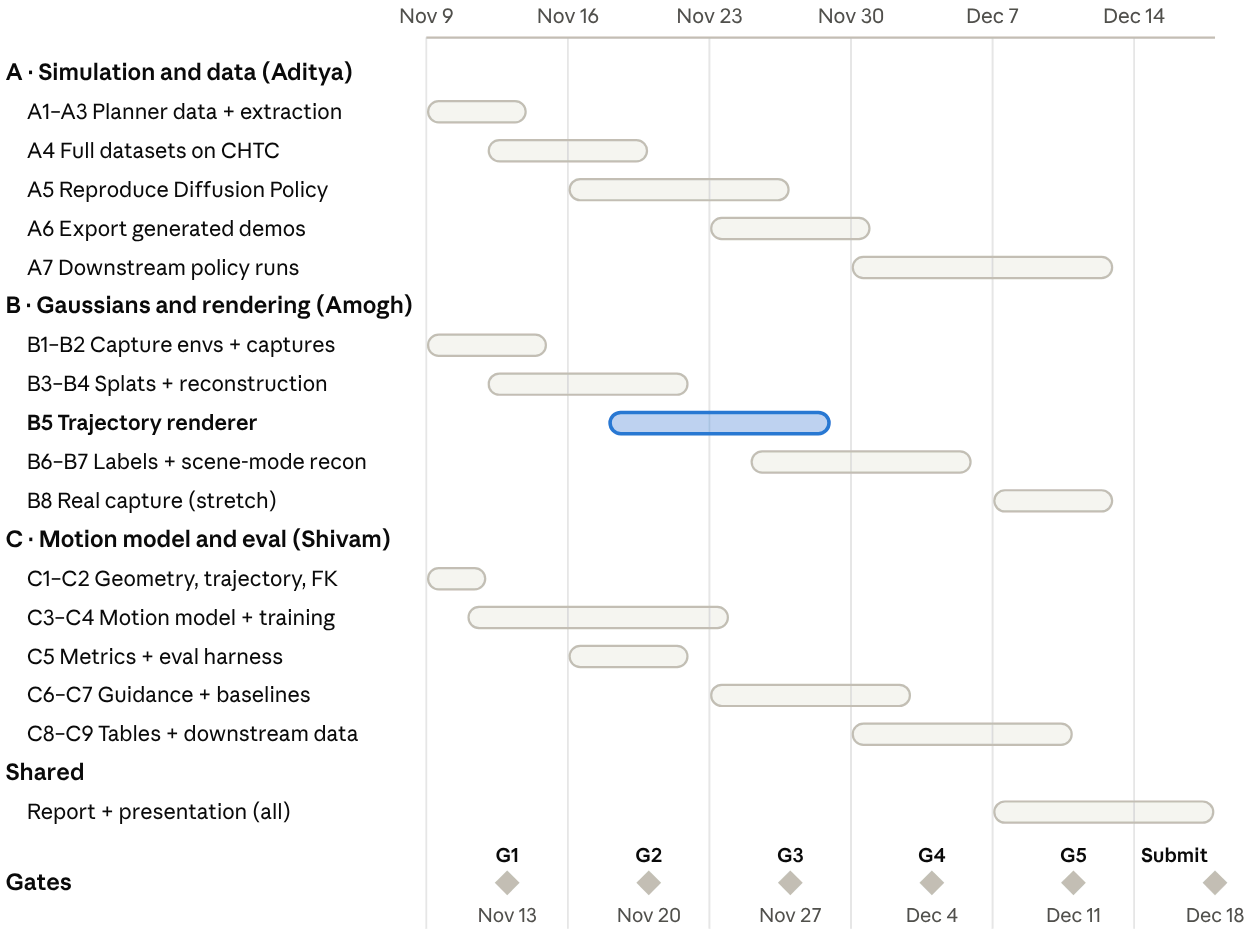
**Each task has:** an ID (A1, B4, C6…), an owner, its dependencies, concrete steps, and an **acceptance check** with a number to hit (for example "held-out PSNR ≥ 28 dB" or "100% replay success on 20 episodes"). A task is done only when the check passes and the numbers are pasted in.

**Workflow:** pick the lowest-numbered task in your workstream whose dependencies are done, set it to *in progress* with your name, work with Claude Code (it reads `CLAUDE.md`), and mark it *done* with the measured numbers.

| Workstream | Owner | Tasks | Main output |
|---|---|---|---|
| Setup (shared) | all | S0.1 repo, S0.2 laptops, S0.3 Docker + CHTC | Everyone can run ManiSkill; GPU jobs work |
| A · Simulation & data | Aditya | A1–A7 | ~20k planner trajectories, Diffusion Policy baseline, exporter, downstream runs |
| B · Gaussians & rendering | Amogh | B1–B8 | Robot + cube + table Gaussians, renderer with labels, render-quality table |
| C · Motion model & evaluation | Shivam | C1–C9 | Trained model, sampler with guidance, baselines, metrics tables, generated demos |

**The renderer (B5) is the critical path into the downstream runs**

Planned work by week, Nov 9 to Dec 18, 2026; diamonds are the weekly gates




6-week plan · 3 workstreams, 5 gates

**Gates** are Friday check-ins: G1 data + first reconstruction + model overfits; G2 model beats the straight-line baseline and renders reach ~28 dB; G3 first full metrics table; G4 downstream results on StackCube; G5 all tables frozen.

**Hand-offs to watch:** C1 (shared geometry code) is needed by A and B in the first days, so interfaces are frozen on day 1; a small data shard goes from A to C at the end of week 1; B's renderer (B5) plus C's sampler feed A's export and policy runs, which is the critical path. If B5 slips past G2, Aditya helps on rendering.

**If we fall behind, cut in this order:** real-world stretch, PickCube downstream runs, the step-by-step baseline, scene-mode reconstruction, then extra policy arms.

# Decisions to confirm

These are the choices the plan makes for us; tick each one you accept, or leave a comment proposing a change, by Sunday, November 8.

### Scope

- [ ] Simulation only for the core results (ManiSkill 3, Franka Panda). Real-world capture is a stretch goal, decided at gate G4.
- [ ] Tasks: StackCube-v1 as the main task, PickCube-v1 for fast development and as a second result.
- [ ] Downstream proof: ManiSkill's Diffusion Policy, RGB input, success rate in simulation. No VLA in v1.
- [ ] Rigid objects only; objects move only when grasped.

### Method

- [ ] Generate poses (robot joints + object poses), not images or Gaussians; Gaussians are built once and moved.
- [ ] All 128 normalised time steps generated jointly, with start and end fixed. The step-by-step version is a baseline, not the method.
- [ ] Goal for downstream data = object goal only (robot's end pose left free).
- [ ] Training motions come from a randomised motion planner (we accept the "why not just use the planner?" question and answer it in the report).

### Evaluation

- [ ] Three baselines: straight line, forward-only, step-by-step, plus the planner as an upper bound.
- [ ] Success criterion: real + generated demos beat real demos alone, and approach real + planner demos.

### Team and schedule

- [ ] Owners: Aditya = simulation & data, Amogh = Gaussians & rendering, Shivam = motion model & evaluation.
- [ ] Six-week schedule, November 9 to December 18, with weekly gates. *Needs the real course deadline.*
- [ ] Compute: CHTC (CPU jobs for data, GPU jobs for Gaussians and training); Colab only for notebooks.
- [ ] Report in the CVPR template, like the proposal.

# What is verified and what is still assumed

The simulation and data side was checked by actually running ManiSkill; the GPU side (Gaussians, model training, policy training) is specified from source code but not yet run.

| Item | Status | How |
|---|---|---|
| Randomised planner works headless on CPU, ~0.5 s per trajectory | Verified | Ran StackCube and PickCube locally with software Vulkan |
| Robot kinematics matches simulator link poses | Verified | Error < 1e-6 m on all 15 links |
| Wrist-camera pose formula | Verified | Matches ManiSkill's own camera to 1e-7 |
| 128 model steps keep enough detail | Verified | 21/21 demos still succeed after converting to 128 steps and back |
| Trajectory file layout, grasp rule, camera setups per task | Verified | Read from source and inspected recorded files |
| gsplat rasterizer API, densification carrying part labels | From source | Read gsplat 1.5.3 code; not yet run on a GPU |
| Reconstruction quality reaches ~28 dB | Assumed | First real check is task B4 |
| Motion model training time (~6–8 h) and quality | Assumed | First real check is C3/C4 |
| Diffusion Policy fits our longer demos in GPU memory | Estimated | ~20 MB per demo, so cap ~600 demos on a 40 GB GPU; check in A5 |
| CHTC Docker image with Vulkan + gsplat builds and runs | Not yet tested | Day-1 task S0.3 |

**Biggest open risk:** whether Gaussian-rendered images are close enough to ManiSkill's renderer that a policy trained on them still works in ManiSkill. The plan measures this directly (render-quality table, and a control arm where our motions are rendered by ManiSkill instead).

## Next steps after review

Once the checklist is settled, we fold any changes into `CLAUDE.md` and `TASKS.md` and start the setup tasks on Monday, November 9.

1. **Sunday, November 8:** everyone comments here; Shivam merges changes into both files.
2. **Monday, November 9 (30 min call):** freeze the interfaces (episode file, part-splats file, render function) and confirm owners.
3. **Monday–Wednesday:** setup tasks S0.1 (repo), S0.2 (each laptop), S0.3 (Docker + CHTC), then each person starts their first workstream task (A1, B1, C1).
4. **Friday, November 13:** gate G1 check-in.